CS3230 Lecture 12

Approximation Algorithms

(ksung@comp.nus.edu.sg)

Lecture Topics and Readings

- Approx algo for the MIN-VERTEX-COVER problem [CLRS] Ch35.1
- Approx algo for the TSP [CLRS] Ch35.2
- Greedy approx algo for the SET-COVER problem [CLRS] Ch35.3
- Polynomial-time approximation scheme for the SUBSET-SUM problem [CLRS] Ch35.5

Although I cannot solve the exact problem, I can approximate it.

Ronald L. Graham

- Mathematician
- Steele Prize for Lifetime Achievement (2003)
  - One of the principal architects of the rapid development worldwide of discrete mathematics in recent years
- Graham (1966) gave the first approximation algorithm on Parallel machine scheduling problem
- Garey, Graham, Ullman, and Johnson are credited to formalize the concept of **polynomial-time approximation algorithm**

Approximation Algorithms

First Example: MIN-VERTEX-COVER

Simple approximation algorithm with constant approximation ratio

Literally Hard Real-life Problems

Many real-life optimization problems are NP-Complete:

- Multi-processor scheduling
- VLSI circuit design
  - Place the component so that it minimizes the production cost, maximizes the energy efficiency
- Transportation
  - Maximizing the number of aircrafts processed by Changi airport
  - Increase the throughput of cargo terminal
- Facility location
  - Place the mobile phone antennas to optimize the coverage
  - Avoid placing hazardous materials near housing
- Supply chain management, etc

Approximation Algorithm

NP-Complete problems are intractable (unless P = NP)

What should we do?

- Solve **small** instances **optimally** (e.g. Lecture 10/DP or Optimized "Brute Force")
- Isolate important **special cases** that are still **optimally** solvable in polynomial time (e.g. Lecture 09/Graph/3 vs 2-CNF-SAT)
- Try to find **near-optimal** solutions in polynomial time (e.g. Lecture 12 today/approximation algorithm)

Today, we study **approximation algorithm** that returns **near-optimal** solution **within a certain ratio**

Approximation Ratio

Optimization problem can be either to minimize or to maximize a (positive) cost

An approximation algorithm **A** is said to have an approximation ratio of **ρ(n)** if for **any** input of size **n**:

- let **C*** be the optimal cost and let **C** be the cost of the solution reported by the approximation algorithm **A**
- Then, **C/C* ≤ ρ(n)** (for minimization) or **C*/C ≤ ρ(n)** (for maximization)
  - We then call algorithm **A** a **ρ(n)**-approximation algorithm
  - Or just a **ρ**-approximation algorithm if the ratio is independent of n

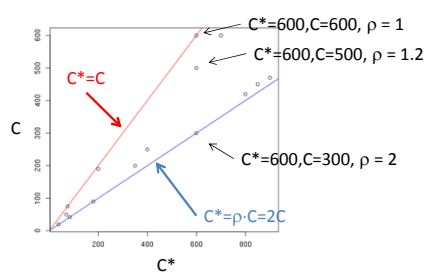
Example (Maximization Problem)

The cost of the solution reported by your algorithm is C

The optimal cost is C*

The approximation ratio = ρ(n) = 2 ≥ max{C*/C}

- It is a 2-approximation algorithm



C*	C	Ratio (C*/C)
17	10	1.7
38	20	1.9
70	50	1.4
75	75	1
84	42	2
180	90	2
200	190	1.052632
350	200	1.75
400	250	1.6
600	300	2
600	600	1
600	500	1.2
700	600	1.166667
800	420	1.904762
850	450	1.888889
900	470	1.914894

MIN-VERTEX-COVER (review)

Input: A graph G = (V, E)

Output: The **<u>minimum</u>** size vertex cover of G

- Definition: Cover C ⊆ V is a vertex cover if for every (u, v) ∈ E, at least one of u and/or v belongs to C

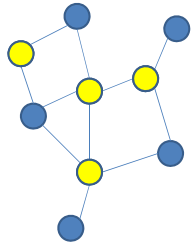
The decision version of VERTEX-COVER is an NP-complete problem (recall the proof in Lecture 08)

- The definition above is the optimization version, which is no easier than the decision version

MIN-VERTEX-COVER – Example

Given a communication network (a graph), we want to place the minimum number of devices (at the vertices) to monitor the performances of all communication links (all the edges)

Answer: The MIN-VERTEX-COVER of this graph is the 4 yellow colored vertices (note that the selected vertices may be not unique but the minimum size = 4 is unique; it is unique in this example though)



Approx-MVC Algorithm

Approx-MVC

```
set C = Φ;
while (E ≠ Φ)
  let (u, v) be any edge in E;
  set C = C ∪ {u, v}; // i.e. both of them
  remove from E all edges incident on u or v;
return C;
```

Running time: The algorithm runs in O(|V|+|E|) time

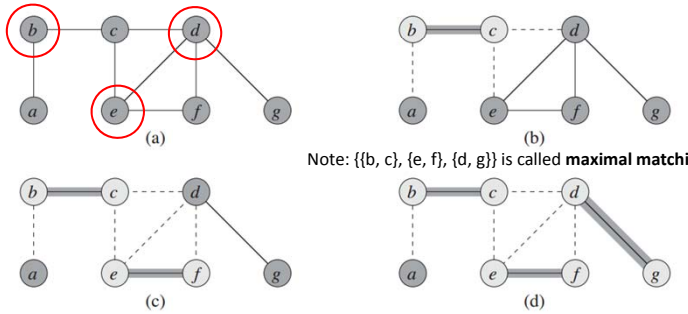
- Classic argument: Each vertex and each edge is visited once

Approx-MVC – Example

Fig (a) is the example graph with min vertex cover C* = {b, d, e}

Fig (b-d) Approx-VC selects edge: (1) {b, c}, (2) {e, f}, (3) {d, g}

Hence, the algorithm selects C = {b, c, d, e, f, g}



Note: {{b, c}, {e, f}, {d, g}} is called **maximal matching**

Approx-MVC – Correctness

Note that the algorithm only removes edges (x, y) from E when either x or y belongs to cover C, i.e. that edge is guaranteed to be covered

- Hence, C is a valid vertex cover

Approx-MVC is a polynomial time **2-approximation** algorithm (1)

```
set C = Φ;
while (E ≠ Φ)
  let (u, v) be any edge in E; // line (#)
  set C = C ∪ {u, v}; // i.e. both of them
  remove from E all edges incident on u or v;
return C;
```

Let **A** be the set of edges picked in the red line (#)

- Note: C = {u, v | (u, v) ∈ A}

Any vertex cover (in particular, the min cover C*) must include at least one endpoint of each edge in A

As no two edges in A share an endpoint, no two edges in A are covered by the same vertex from C*, i.e. |A| is the **lower bound** of the size of the min cover C*

- |C*| ≥ |A|

Approx-MVC is a polynomial time **2-approximation** algorithm (2)

```
set C = Φ;
while (E ≠ Φ)
  let (u, v) be any edge in E; // line (#)
  set C = C ∪ {u, v}; // i.e. both of them
  remove from E all edges incident on u or v;
return C;
```

In every iteration of Approx-MVC, we pick an edge for which neither of its endpoints is already in C

- Therefore, |C| = 2*|A|, 2 times the number of edges picked
- Or |A| = |C| / 2

Hence, |C*| ≥ |A|, or |C*| ≥ |C| / 2, or |C| ≤ 2 * |C*|

- Thus the approximation ratio is 2 (Question: Is this good?)

Summary of the Approach

Note that we do not even know the optimal solution

- In MIN-VERTEX-COVER, we don't know the min cover |C*|

But by estimating the bound of the optimal solution, we can still get the approximation ratio

- For example, in MIN-VERTEX-COVER, we use the set A to get the lower bound of the MIN-VERTEX-COVER

Approximability of NPC Problems

Can we get **constant**-approximation algorithm for all NP-complete problems like MIN-VERTEX-COVER?

- The answer: Unfortunately no ☹

Different NPC problems have different *approximability*:

1. **ρ**-approximation or **ρ(n)**-approximation ratio, e.g. MIN-VERTEX-COVER, TSP-METRIC (both 2-approx), and later SET-COVER ((lg |X|+1)-approx)
2. No-finite approximation ratio, e.g. TSP
3. Polynomial-time approximation scheme, i.e. (1+ε)-approx

We will see more examples later.

Approximation Algorithms

Second Example: Two cases for TSP

Approximation is possible for some NPC problems.
However, some problems are inapproximatible

TRAVELING-SALESMAN-PROBLEM (TSP)

Input:

- A complete undirected graph **G = (V, VxV)**
- A nonnegative cost function **c:VxV→Z**
- A cost threshold **k**

Output: A tour that visits all vertices in **V** with the **minimum** cost

The decision version of this problem is NP-complete (see proof in Lecture 08)

Additional Assumption: Δ Inequality

Triangle (Δ) Inequality: $c(u, w) \leq c(u, v) + c(v, w)$ for all vertices $u, v, w \in V$

- This assumption is valid for most of TSP applications (e.g. visiting all cities in US)

TSP with this additional assumption (TSP-METRIC) is still NP-Complete (exercise in tutorial)

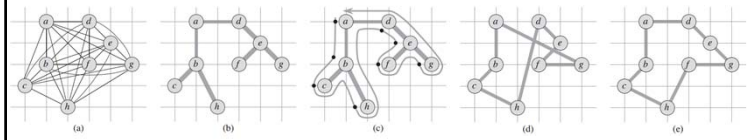
Can we find a good approximation algorithm?

Approx-TSP-TOUR

Algorithm Approx-TSP-Tour(G)

1. Compute a Minimum Spanning Tree **T** for **G**
2. Let **L** be the list of vertices visited in preorder tree walk of **T**
3. Return Hamiltonian cycle **H** that visits the vertices in the order of **L**, skipping repeated vertices

We illustrate the steps below (G is on a 2-D grid):



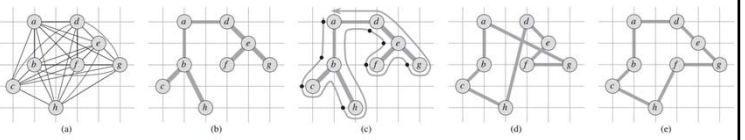
Input | Step 1 | Step 2 | Step 3 Cost = 19.074 | Optimal Cost = 14.715

Approx-TSP-Tour runs in polynomial time

Step 1: Compute a Minimum Spanning Tree T for G can be done in polynomial time, i.e. in **O(E log V)** using either Prim's or Kruskal's algorithm

Step 2+3: Preorder traversal is **O(V)**

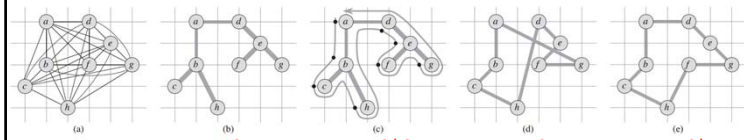
Therefore Approx-TSP-Tour runs in polynomial time



Input | Step 1 | Step 2 | Step 3 Cost = 19.074 | Optimal Cost = 14.715

Approx-TSP-Tour is a 2-approx algorithm

- Let **C*** be the cost of the optimal tour
- Let $\mathbf{C_T}$ be the sum of cost of edges in the MST **T**
- The optimal cost $\mathbf{C^*} \geq \mathbf{C_T}$ (that lower bound argument again)
- Let $\mathbf{C_H}$ be the sum of cost of the Hamiltonian cycle **H**
- By Triangle inequality, we have $\mathbf{C_H \leq 2 * C_T}$
- Hence, $\mathbf{C_H \leq 2 * C_T \leq \underline{2} * C^*}$
- Thus the approximation ratio = 2



C_T | $2*C_T$ | C_H | C^*

General TSP

TSP-<u>METRIC</u> has a 2-approximation polynomial time algorithm, but how about TSP in general?

- Answer: We cannot use the previous approximation algorithm since the triangle inequality may not hold

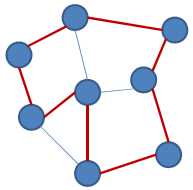
Strong claim: We claim that there is no polynomial time constant-approximation algorithm that can solve the general form of TSP (this also completes the Nearest Neighbor Heuristic discussion in Lecture 11)

- Idea: Show that HAM-CYCLE $\leq_p$ ρ-approx_TSP

Review: HAM-CYCLE problem

Input: Graph G = (V, E)

Output: yes if a Hamiltonian cycle exists in G

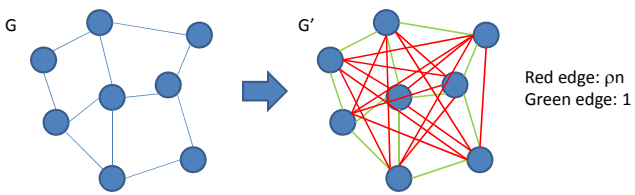


HAM-CYCLE $\leq_p$ ρ-approx_TSP

Given G = (V, E) where n = |V|, construct G' = (V, VxV)

- For every u, v ∈ V, $c(u, v) = \begin{cases} 1 & if\ (u,v) \in E \\ \rho n + 1 & otherwise \end{cases}$
  - Note that this reduction is clearly polynomial, O(V + E)

Correctness claim: G has a Hamiltonian-cycle iff G' has a tour of cost at most ρn



G has a Hamiltonian-cycle iff G' has a tour of cost at most ρn

If G contains a Hamiltonian cycle
H = (v_1, v_2, v_3, …, v_{n-1}, v_n, v_1),

- Then, the cost the tour H in G' is
  $c(v_1, v_2) + c(v_2, v_3) + \ldots + c(v_{n-1}, v_n) + c(v_n, v_1) = n$, which is ≤ ρn

If G does not contain a Hamiltonian cycle,

- Any tour in G' contains at least one edge not in E, which costs a <u>heavy penalty</u> of ρn+1
- Then, the cost of the tour is <u>at least</u> ρn+1 > ρn

No polynomial-time constant approximation algorithm for the general TSP

We have shown that HAM-CYCLE $\leq_p$ ρ-approx_TSP

- If ρ-approx_TSP can be solved in polynomial time, then HAM-CYCLE can also be solved in polynomial time
- We arrive at contradiction compared to our result in Lecture 08 (that HAM-CYCLE is NP-Complete)

Theorem: If P ≠ NP, there is **<u>no</u>** polynomial-time constant approximation algorithm for the general TSP

General technique for proving that a problem cannot be approximated well

Consider a minimization problem Y

To show that Y cannot be approximated well, we reduce an NP-complete (decision) problem X to the (decision version of) approx_Y

- "yes" instances of X correspond to instances of Y with value at most k (for some k)
- "no" instances of X correspond to instances of Y with value greater than ρk

Then we conclude, unless P = NP, there is no polynomial time ρ-approximation algorithm for Y

Approximation Algorithms

Third Example: SET-COVER

(ln |X|+1)-approximation algorithm

SET-COVER problem

Input: (X, F) where X is a finite set and F is a family of subsets of X such that every element of X belongs to at least one subset in F, i.e. X = $\bigcup_{S \in F} S$

Output: Find the <u>minimum</u> size set cover C (opt version)

- Definition: C $\subseteq$ F is called a set cover if X = $\bigcup_{S \in C} S$

Example:

- X = { 1, 2, 3, 4, 5, 6, 7, 8, 9 }
- F = { {2, 3, 4, 5, 6}, {1, 2, 3, 4}, {4, 5, 6, 7}, {5, 6, 7, 8}, {8, 9}, {1} }
- C = { {1, 2, 3, 4}, {5, 6, 7, 8}, {8, 9} }, of size = 3 sets

"Greedy" Approximation Algorithm

Algorithm Greedy-SC(X, F)

```
set U = X;
set C = Φ;
while (U ≠ Φ)
   select S ∈ F that maximizes |S ∩ U| // greedy
   U = U – S;
   C = C ∪ {S};
```

This "greedy" algorithm picks set S $\in$ F that covers the max number of remaining elements that are uncovered (we break ties arbitrarily)

- There are several possible implementation options, the easiest is to simulate the loop body in O(|X|*|F|) and notice that the loop will not run more than O(min(|X|, |F|))

Example

- X = { 1, 2, 3, 4, 5, 6, 7, 8, 9 }
- F = { ~~{2, 3, 4, 5, 6}~~, {1, 2, 3, 4}, {4, 5, 6, 7}, ~~{5, 6, 7, 8}~~, ~~{8, 9}~~, ~~{1}~~ }

1. First, select S_1 = {2, 3, 4, 5, 6} since it is the biggest set (greedy)
2. Second, we can select either {5, 6, 7, 8} or {8, 9}, WLOG, we select S_2 = {5, 6, 7, 8}
3. Third, we can select either {8, 9} or {1}, WLOG, we select S_3 = {8, 9}
4. Fourth, we select S_4 = {1}
5. Hence, C = { S_1 = {2, 3, 4, 5, 6}, S_2 = {5, 6, 7, 8}, S_3 = {8, 9}, S_4 = {1} }, of size = 4 sets
6. Note that C* = { {1, 2, 3, 4}, {5, 6, 7, 8}, {8, 9} }, of size = 3 sets

S_4	x								
S_3								x	x
S_2					x	x	x	x	
S_1		x	x	x	x	x			
	1	2	3	4	5	6	7	8	9

Analysis of Greedy-SC

Theorem: Greedy-SC is a polynomial-time H(m)-approximation algorithm, where m = max{|S|: S∈F}

- Harmonic number: $H(d) = H_d = \sum_{i=1}^{d} 1/i$

Proof (a bit complex, hang on):

- Let S_i be the ith subset selected by Greedy-SC (cost = 1 set)
- We spread this 1 unit cost evenly to all elements that are covered for the first time, i.e. elements $S_i - (S_1 \cup S_2 \cup \ldots \cup S_{i-1})$
- Suppose x ∈ X is covered for the first time by S_i
  The cost assigned to x is $c_x = \frac{1}{|S_i - (S_1 \cup S_2 \cup \ldots \cup S_{i-1})|}$
- Note that the size of the set cover $|C| = \sum_{x \in X} c_x$

Previous example (approx score)

For the previous example, we select S_1, S_2, S_3, and S_4, so:

- $c_2 = c_3 = c_4 = c_5 = c_6 = 1/5$ since they first appear in S_1
- $c_7 = c_8 = 1/2$ since they first appear in S_2
- $c_9 = 1$ since it first appears in S_3
- $c_1 = 1$ since it first appears in S_4

Hence, the size of the set cover $|C| = \sum_{x \in X} c_x$ = 4 sets

	1	1/5	1/5	1/5	1/5	1/5	1/2	1/2	1
S_4	x								
S_3								x	x
S_2					x	x	x	x	
S_1		x	x	x	x	x			
	1	2	3	4	5	6	7	8	9

Score for optimal

Let C* be the optimal (min size) set cover

Since each x ∈ X is in at least one set in C*, the cost assigned to the optimal set cover is

$$\sum_{S \in C^*} \sum_{x \in S} c_x \geq \boxed{\sum_{x \in X} c_x = |C|}$$

or

$$|C| \leq \sum_{S \in C^*} \sum_{x \in S} c_x$$

Previous example (optimal score)

C* = { S_2 = {1, 2, 3, 4}, S_4 = {5, 6, 7, 8}, S_5 = {8, 9} }

$$\sum_{S \in C^*} \sum_{x \in S} c_x =$$

(1+1/5+1/5+1/5) + (1/5+1/5+1/2+1/2) + (1/2+1) = $4\frac{1}{2}$

This is $\geq |C|$ = 4

{8,9}								1/2	1
{5,6,7,8}					1/5	1/5	1/2	1/2	
{1,2,3,4}	1	1/5	1/5	1/5					
	1(1)	2(1/5)	3(1/5)	4(1/5)	5(1/5)	6(1/5)	7(1/2)	8(1/2)	9(1)

Summary of Greedy-SC Analysis

In the appendix (complex), we show that $\sum_{x \in S} c_x \leq H(|S|)$

Hence, $|C| \leq \sum_{S \in C^*} \sum_{x \in S} c_x \leq \sum_{S \in C^*} H(|S|) \leq |C^*| H(m)$

Thus, the approximation ratio is H(m) ≤ ln m+1 ≤ ln|X|+1

Implication: As |X| grows, the size of the approximate solution Greedy-SC grow, relative to the size of an optimal solution by at most ln |X|+1 time

But as logarithm grows very slowly, this algorithm is still useful

Summary

Today, we have seen:

1. ρ-approximation or ρ(n)-approximation ratio, e.g. MIN-VERTEX-COVER, TSP-METRIC (both 2-approx), and SET-COVER ((lg |X|+1)-approx)
2. No-finite approximation ratio, e.g. TSP

Not discussed today:

3. Polynomial-time approximation <u>scheme</u>, i.e. (1+ε)-approx

Some non-optimal algorithms for NPC problems do not need good approximation ratio to be useful… (Week13)

Appendix: Proof of $\sum_{x \in S} c_x \leq H(|S|)$ (I)

- Let $u_i = |S - (S_1 \cup S_2 \cup \ldots \cup S_i)|$ for i ≥ 1 and $u_0 = |S|$
- Note that u_i is monotonic decreasing
- Let k be the smallest index such that $u_k = 0$
- Then, $u_{i-1} - u_i$ elements of S are covered for the first time by S_i
- Hence, $\sum_{x \in S} c_x = \sum_{i=1}^{k} (u_{i-1} - u_i) \cdot \frac{1}{|S_i - (S_1 \cup \ldots \cup S_{i-1})|}$
- Observe that $|S_i - (S_1 \cup \ldots \cup S_{i-1})| \geq |S - (S_1 \cup \ldots \cup S_{i-1})| = u_{i-1}$
- Thus, $\sum_{x \in S} c_x \leq \sum_{i=1}^{k} (u_{i-1} - u_i) \cdot \frac{1}{u_{i-1}}$

Appendix: Proof of $\sum_{x \in S} c_x \leq H(|S|)$ (II)

$$
\begin{aligned}
\sum_{x \in S} c_x &\leq \sum_{i=1}^{k} (u_{i-1} - u_i) \cdot \frac{1}{u_{i-1}} \\
&= \sum_{i=1}^{k} \sum_{j=u_i+1}^{u_{i-1}} \frac{1}{u_{i-1}} \\
&\leq \sum_{i=1}^{k} \sum_{j=u_i+1}^{u_{i-1}} \frac{1}{j} && \text{(because } j \leq u_{i-1}\text{)} \\
&= \sum_{i=1}^{k} \left(\sum_{j=1}^{u_{i-1}} \frac{1}{j} - \sum_{j=1}^{u_i} \frac{1}{j} \right) \\
&= \sum_{i=1}^{k} (H(u_{i-1}) - H(u_i)) \\
&= H(u_0) - H(u_k) && \text{(because the sum telescopes)} \\
&= H(u_0) - H(0) \\
&= H(u_0) && \text{(because } H(0) = 0\text{)} \\
&= H(|S|)\,,
\end{aligned}
$$

Polynomial time approximation scheme

Fourth Example: Subset-sum

We can approximate with an arbitrarily small approximation ratio

Polynomial time approximation scheme

- All previous approximation algorithms have fixed approximation ratio (say, a fixed constant).
- When there exists an approximation algorithm whose approximation ratio is $(1+\varepsilon)$ for any $\varepsilon>0$,
  - Such approximation scheme is called a **polynomial-time approximation scheme (PTAS)**.

Fully polynomial-time approximation scheme

- As ε decreases, the running time of PTAS increases.
- Normally, the running time is exponential in $1/\varepsilon$ (e.g. $O(n^{1/\varepsilon})$).
- If the running time of the PTAS is polynomial in both $1/\varepsilon$ and n (e.g $O((1/\varepsilon)^2 n^3)$),
  - it is called a **fully polynomial-time approximation scheme**.

Subset-sum problem

- Input: A set of integers S and an integer t.
- Output: S' that maximizes $\sum_{x\in S'} x$ and $\sum_{x\in S'} x \le t$.
- Example:
  - S={3, 7, 8, 11, 13}, t=17
  - Ans: S'={13, 3}

Exponential-time exact algorthm

- Algorithm Exact-Subset-SUM(S,t)
  - Set n = |S|;
  - Set P_0 = {0};
  - For i=1 to n
    - $P_i = P_{i-1} \cup \{y+x_i \mid y \in P_{i-1}, y+x_i \le t\}$;
  - Return the largest element in P_n;
- The algorithm maintains P_i as a sorted list of the sums of all subsets of $\{x_1,\ldots,x_i\}$.

Example

- S = {1 , 4, 5}, t=7.
  - P_0 = {0}
  - P_1 = {0}∪{1} = {0,1}
  - P_2 = {0,1}∪{4,5} = {0,1,4,5}
  - P_3 = {0,1,4,5}∪{5,6} = {0,1,4,5,6}
  - Return 6 (the largest element in P_3)

Idea of trimming

- P_i can be very long.
- For the purpose of finding approximate solution, if two values in P_i are close, we can remove one.
- Precisely, let δ be a trimming parameter (0<δ<1).
- We can remove an element y from P_i if there exists z such that $\frac{y}{1+\delta} \le z \le y$.
- For example,
  - P_i={0, 1, 10, 11, 12, 15, 20, 21, 22, 23, 24, 29}.
  - For δ=0.1, we can trim L and obtain P_i'={0, 1, 10, 12, 15, 20, 23, 29}
    - 11 is represented by 10; 21,22 are represented by 20; 24 is represented by 23.

Algorithm Trim

- Algorithm trim({y_1, y_2, …, y_m}, δ)
- /* Assume $y_1<y_2<\ldots<y_m$ */
  - Set Q={y_1};
  - z = y_1;
  - For i = 2 to m
    - If $\frac{y_i}{1+\delta} \le z$ then /* note: $z \le y_i$ */
      - /* Remove y_i */
    - else
      - Q = Q ∪ {y_i};
      - Set z = y_i;
  - Return Q;
- Running time: O(m)

Example

- Trim({0, 1, 10, 11, 12, 15, 20, 21, 22, 23, 24, 29}, δ=0.1)

- Q={0}
- Remove $y_i \le 0(0.1)=0$
- Q={0,1}
- Remove $y_i \le 1(1.1)=1.1$
- Q={0,1,10}
- Remove $y_i \le 10(1.1)=11$
- Q={0,1,10,12}
- Remove $y_i \le 12(1.1)=13.2$
- Q={0,1,10,12,15}
- Remove $y_i \le 15(1.1)=16.5$
- Q={0,1,10,12,15,20}
- Remove $y_i \le 20(1.1)=22$
- Q={0,1,10,12,15,20,23}
- Remove $y_i \le 23(1.1)=25.3$
- Q={0,1,10,12,15,20,23,29}

Size of the list after trimming

- **Lemma**: Assume the values in L are at most t. After trim(L, δ), the list contains at most $2+\log_{(1+\delta)}t$ values.

- Proof:
- After trimming, successive elements z and z' in the trimmed list must satisfy
  - (1) $z, z' \le t$
  - (2) $z'/z > 1 + \delta$.
- Hence, after 0 and 1, the kth value in the list should be bigger than $(1+\delta)^k$.
- L contains possibly the values 0 and 1 and up to $\log_{1+\delta} t$ additional values.

Approx-Subset-SUM

- Algorithm Approx-Subset-SUM(S,t,ε)
  - Set n = |S|;
  - Set L_0 = {0};
  - For i=1 to n
    - $K_i = L_{i-1} \cup \{y+x_i \mid y \in L_{i-1}, y+x_i \le t\}$;
    - L_i = Trim(K_i, $\varepsilon/(2n)$);
  - Return the largest element in L_n;

Example

- Input: S={104, 102, 201, 101}, t=308, ε=0.4 (δ= $\varepsilon/(2n)$= $\varepsilon/8$=0.05)

- L_0={0}
- (Before trim) K_1={0}$\cup${104}={0,104}
- (After trim) L_1 = {0,104}
- (Before trim) K_2={0,104}$\cup${102,206}={0,102,104,206}
- (Afer trim) L_2={0,102,206}
- (Before trim) K_3={0,102,206}$\cup${201,303}={0,102,201,206,303}
- (After trim) L_3={0,102,201,303}
- (Before trim) K_4={0,102,201,303}$\cup${101,203,302}={0,101,102,201,203,302,303}
- (After trim) L_4={0,101,201,302}
- Report 302

- Note: The optimal value is 307=101+102+104, which is within 302(1+0.4).

Running time of Approx-Subset-SUM

- The running time is $O(\sum_{i=1..n} |L_i|)$.
- Note that after trimming, L_i has at most $\log_{1+\varepsilon/(2n)} t + 2 = \frac{\ln t}{\ln(1+\frac{\varepsilon}{2n})} + 2 \leq \frac{2n\left(1+\frac{\varepsilon}{2n}\right)\ln t}{\varepsilon} + 2 \leq \frac{4n \ln t}{\varepsilon} + 2.$
- Thus, the running time is $O\left(n\left(\frac{4n \ln t}{\varepsilon} + 2\right)\right) = O\left(\frac{4n^2 \ln t}{\varepsilon}\right)$.

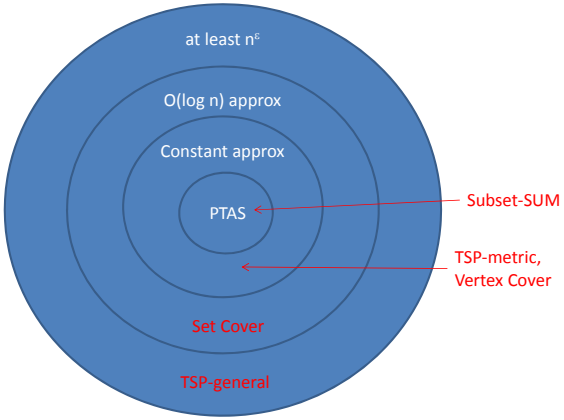
Lemma: For every element $y \in P_i$, there is a $z \in L_i$ such that $\frac{y}{(1+\frac{\epsilon}{2n})^i} \leq z \leq y$.

- We prove the lemma by induction.
- For i=0, $L_0 = P_0 = \{0\}$. The lemma is true.
- Assume the lemma is true for i.
- I.e., for every element $y \in P_i$, there is a $z \in L_i$ such that $\frac{y}{(1+\frac{\epsilon}{2n})^i} \leq z \leq y$.
- Recall that
  - $P_{i+1} = P_i \cup \{y+x_{i+1} \mid y \in P_i, y+x_{i+1} \leq t\}$
  - $K_{i+1} = L_i \cup \{y+x_{i+1} \mid y \in L_i, y+x_{i+1} \leq t\}$
- For every element $y' \in P_{i+1}$,
  - If $y' \in P_i$, we have $z \in L_i \subseteq K_{i+1}$ such that $\frac{y'}{(1+\frac{\epsilon}{2n})^i} \leq z \leq y'$.
  - If $y' \in \{y+x_{i+1} \mid y \in P_i, y+x_{i+1} \leq t\}$, we have $z+x_{i+1} \in K_{i+1}$ such that $\frac{y'}{(1+\frac{\epsilon}{2n})^i} \leq z + x_{i+1} \leq y'$.
- In other word, for every element $y \in P_{i+1}$, there is a $z \in K_{i+1}$ such that $\frac{y}{(1+\frac{\epsilon}{2n})^i} \leq z \leq y$.
- As L_{i+1}=trim(K_{i+1}, $\varepsilon/(2n)$), for every $z \in K_{i+1}$, there is $z' \in L_{i+1}$ such that $\frac{z}{1+\frac{\epsilon}{2n}} \leq z' \leq z$.
- Hence, for every element $y \in P_{i+1}$, there is a $z' \in L_{i+1}$ such that $\frac{y}{(1+\frac{\epsilon}{2n})^{i+1}} \leq z' \leq y$.
- By induction, the lemma follows.

The approximation ratio of Approx-Subset-SUM is 1+ ε

- Let y* be the value reported by Exact-Subset-SUM (i.e. optimal solution).
- Note that y* is the biggest value in P_n.
- By the previous lemma, there is a $z \in L_n$ such that $\frac{y^*}{(1+\frac{\epsilon}{2n})^n} \leq z \leq y^*$.
- Let z* be the value reported by Approx-Subset-SUM.
- Note that z* is the largest value in L_n.
- Hence, $\frac{y^*}{z^*} \leq \left(1+\frac{\epsilon}{2n}\right)^n \leq e^{\frac{\epsilon}{2}} \leq 1 + \frac{\epsilon}{2} + \left(\frac{\epsilon}{2}\right)^2 \leq 1 + \epsilon.$
- Thus, the approximation ratio is 1+ ε.

Approx-Subset-SUM is a fully polynomial-time approximation scheme

- Running time: $O\left(\frac{4n^2 \ln t}{\varepsilon}\right)$.
- Approximation ratio: 1+ ε.
- Hence, Approx-Subset-SUM is a fully polynomial-time approximation scheme.

Approximatibility